

The idea of Dev/Ops

Modern DevOps for Systems Biologists

Jun Allard

Why systems biologists need Dev/Ops now

Motivation 1 of 4: The elevation of computational and mathematical biology

A realization that dawned on me in March 2020.

Recent worldwide events have challenged the mathematical biology community.

- ▶ A global pandemic put mathematical biology, and mathematical biologists, on the world stage — predictive epidemiology, in front of policy decision-makers.
- ▶ To keep that impact and respect, our standards of rigor in **coding** have to be elevated to match our standards of rigor in **mathematics**.
- ▶ The gap is troubling, because mathematical rigor has been a point of pride in the field. It is also an opportunity.



I'm conscious that lots of people would like to see and run the pandemic simulation code we are using to model control measures against COVID-19. To explain the background - I wrote the code (thousands of lines of undocumented C) 13+ years ago to model flu pandemics...

2:13 PM · Mar 22, 2020 · [Twitter for iPhone](#)

1.4K Retweets 4.9K Likes



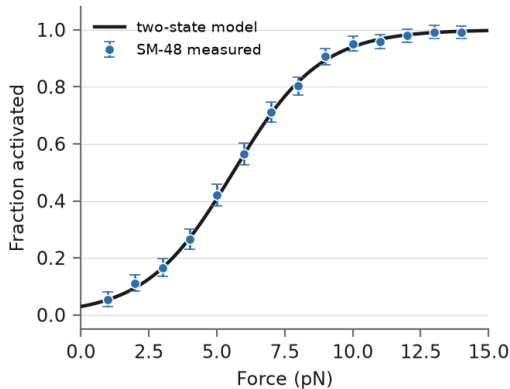
neil_ferguson  @neil_ferguson · Mar 22

Replying to @neil_ferguson

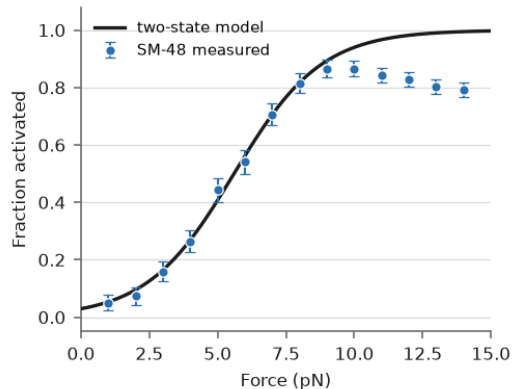
I am happy to say that @Microsoft and @GitHub are working with @Imperial_JIDEA and @MRC_Outbreak to document, refactor and extend the code to allow others to use without the multiple days training it would currently require (and which we don't have time to give)...

Motivation 2 of 4: A case where version control saved me

The plot I had



The plot I could reproduce



An early grad student convinced me to use version control: github.com/allardjun/a-typical-paper.

Find the quote from Twitter in 2019 from the grad student.

Motivation 3 of 4: The DevOps approach to science

Software development

Develop, operate, develop, operate, develop, operate.



Ship something that runs, then make it better.



Building a car

Motor, chassis, wheels, frame, ...



Or: first build a bicycle.



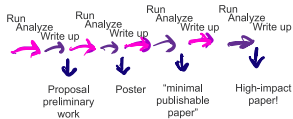
Henrik Kniberg

Graduate school

Bench phase, then coffee-shop phase.



Or: a little of both, every week.



The DevOps approach is to make **small-batch, continuous, end-to-end** improvements.

Get the *Nature* editorial capturing this idea.

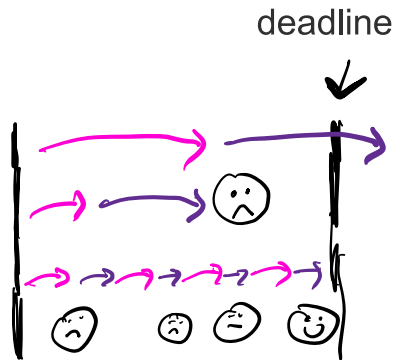
Pros and cons of working this way

Cons

- ▶ **Wasteful.** You don't use a lot of what you do. Think of the industrial revolution and assembly lines.
- ▶ **Misses the advantages of scale** that come from doing things in parallel.
- ▶ **Might invalidate statistical design** — consider a randomized control trial.
- ▶ **A lot of time spent keeping different versions organized.**

Pros

- ▶ For any complex, uncertain project (i.e., science!), **all the parts of the project gradually adapt** to achieve the goal.
- ▶ **Reduces time-to-deadline uncertainty.** Too slow or too unambitious? No problem.



Motivation 4 of 4: Supervising powerful-but-unreliable team members

- ▶ At early career stages, **you write the code**.
- ▶ For many career paths — including outside science, and outside coding — you eventually **mentor, coach and supervise others**.
- ▶ Your job becomes helping them, critically testing their approaches, and catching bugs in their thinking.
- ▶ You need a tool that lets them **say what they are thinking**, lets you **revert** if necessary, and gives you an opportunity to **look over what they are contributing**.

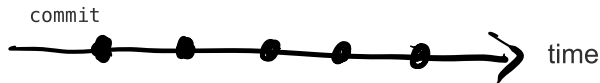
Coding yourself, supervising a team, supervising agentic coders

figure not drawn yet - svg_supervising.svg

In the age of agentic coders, the skill of **supervising** needs to be learned at earlier career stages.

Working with your past and future self

Version control with git



Advantage: **bisection search** for trouble. When something broke between here and there, git will help you halve the interval until you find it.

Vocabulary

- ▶ repo
- ▶ commit
- ▶ stage / add
- ▶ commit message
- ▶ commit hash
- ▶ HEAD
- ▶ checkout

Two different things

- ▶ **git** — an open-source technology
- ▶ **GitHub** — a company

How often should you commit?

Ask what the action you just took actually was.

- ▶ Running something to get a result — `experiment`
- ▶ Adding a feature, mode or setting — `feat`
- ▶ Fixing a bug or troubleshooting — `bugfix`
- ▶ Refactoring, cleaning up or improving without adding new capabilities — `refactor`

A commit should not include more than one of those.

Committing is the one opportunity to slow down and think. Avoid automatic commits – this is not the time.

git is already there

Git is already built in to almost all the software you interact with.

- ▶ **VSCode, Jupyter, MATLAB** — full support, already installed.
- ▶ **Microsoft Office, Mathematica** — it works, but misses some features.

Overleaf (latex)

Matlab

Jupyter

The image is a collage of three screenshots from different software applications:

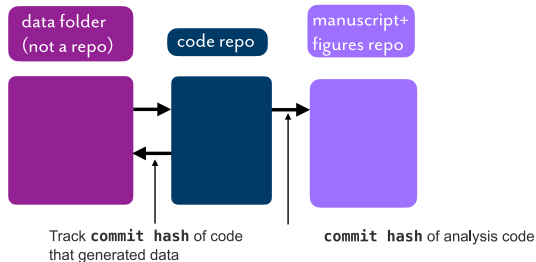
- Overleaf (latex):** The top-left screenshot shows the Overleaf web interface. It includes a navigation bar with 'Download', 'Actions' (Copy Project, Word Count), 'Sync' (Dropbox, Git, GitHub), and 'Settings' (Compiler: pdfLaTeX). The main content area shows a LaTeX document with a table of contents and a list of files.
- Matlab:** The top-middle screenshot shows the MATLAB R2014b desktop environment. It features a 'HOME' tab with 'New', 'Open', 'Save', 'Find Files', 'Compare', 'Go To', 'Comment', 'Indent', 'Find', and 'Print' buttons. The 'Current Folder' pane on the left shows a directory structure with files like 'Algorithms', 'Colormaps', 'infern.m', 'magma.m', 'plasma.m', 'wind.m', 'demo1.m', 'demo2.m', 'README.txt', 'Data_created', 'Demos', 'Documentation', 'Helical', 'Mex_files', 'Motion_paper_scripts', 'Not_used', 'random_images', 'Real_data', 'Source', 'Test_data', 'Third_party_tools', 'Utilities', 'Deformation_Vector_Fields', and 'Quality_measures'. The 'Editor' pane on the right shows a script named 'plotmg.m' with MATLAB code.
- Jupyter:** The top-right screenshot shows the Jupyter web interface. It includes a navigation bar with 'master' and 'Math227C / ProblemSets_PartII / Math227C20Sp_P09'. The main content area shows a commit message 'allardjun Bootstrap minor edits to text' and a contributor list. Below this, there is a section titled 'Problem Set 9' with the subtitle 'Example: predicting colon cancer from str'. The 'In []:' section shows a Jupyter cell with the following code:

```
# plot settings
options(repr.plot.width=15, repr.plot.height=10)

In [24]: # Install a package BioConductor ExperimentHub to ac
if (!requireNamespace("BiocManager", quietly = TRUE))
  install.packages("BiocManager")
BiocManager::install()
BiocManager::install("ExperimentHub")

# Install glmnet for LASSO and Elastic Net regressio
install.packages("glmnet")
```

Directory architecture and large data



How we (the Allard group) organize a project — **three places**:

1. **Large files**, outside of git, backed up (e.g., Google Drive, Dropbox, linux server rack).
2. **Code**, in a git repo.
3. **Manuscript and figures**, in another git repo (not shared).

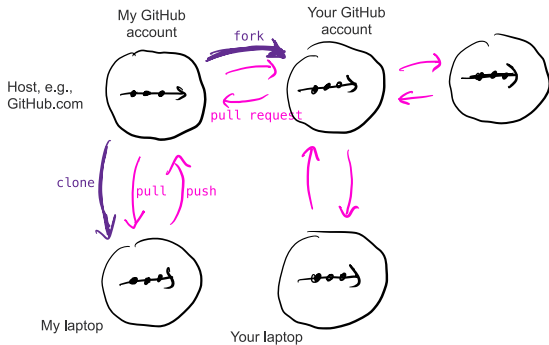
The three locations share each other's commit hashes, so every figure knows which commit it came from.

You *can* put large files in the git repo and reach for `.gitignore` instead — but the three-place split is what has held up.

Version control with git can be used smoothly for entire projects — analysis and write-ups, not just code. That is what lets you take a continuous-improvement “DevOps” approach, with the pros and cons above.

Working with a team

git fork, clone, branch, merge, pull request



In the cloud

GitHub, and other providers: companies, university servers, ...

- ▶ push, pull
- ▶ clone
- ▶ fork
- ▶ pull request

The idea of **repo ownership**: taking is free, giving requires permission.

Advantages *and* disadvantages over automatic history, like Apple Time Machine or Google Docs.

Much we did not discuss

There is much we did not discuss.

- ▶ Branching
- ▶ How to go back to a previous version
- ▶ How to deal with merge conflicts
- ▶ Cherry-picking

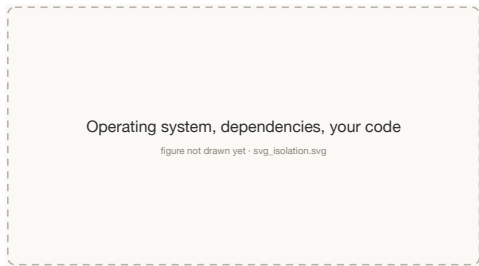
Working with the world, across teams and time

Reproducibility, dependency, security and isolation

For your code to impact the scientific community, it should work across teams, computers, and dependency upgrades.

- ▶ One definition of the difference between a **software developer** and a **coder**: making code that works across teams and time — that is, handling dependencies.
- ▶ Your dependencies **will** get updates. Some you really want, e.g. security fixes. Some you cannot avoid, e.g. operating system updates.
- ▶ *“But it worked on my machine.”* Continually assuring that it works on **any** machine, for **any** person, without you helping, is how you live up to the DevOps philosophy. It is a huge mental-health gain: it eliminates the “heroics” and “heist mode” at the end of a project.
- ▶ One project should not screw up another project, even on the same machine. One actor — human? agent? — should not screw up another actor, even on the same machine.

Three levels of isolation



- ▶ **Virtual machines** — replicate the operating system.
- ▶ **Containers** — replicate and isolate the file structure, but share the operating system kernel.
- ▶ **Environments** — replicate and isolate the package dependencies.

Environments in Python, Conda, Julia, R and Matlab

Same idea everywhere: make a local copy of your dependencies, activate it, and *record* it so someone else can rebuild it.

Python

- ▶ Older way: a `venv`, memorialized in `requirements.txt`; someone else recreates it with `pip install -r requirements.txt`.
- ▶ Newer way: `uv.lock`; someone else recreates it with `uv run`.

Conda

- ▶ Make one and switch to it: `conda create -n myproject`, then `conda activate myproject`.
- ▶ Memorialized in `environment.yml`, written by `conda env export`; someone else recreates it with `conda env create`.

Julia

- ▶ All dependencies needed by any project on this machine live in `$JULIA_DEPOT`.
- ▶ Each project records what it needs in `Project.toml` and `Manifest.toml`.
- ▶ Someone else recreates your environment from those.

Complete the Conda description, and decide whether R and Matlab get their own column or come out of the title.

Add the quick code snippet that shows one project with one version of `numpy` and one definition of `inf`, and a sibling directory with a different version and different `inf` behaviour.

Confirm the Julia command that recreates the environment: `Pkg.instantiate()`.

Containerization: Docker and dev-containers

The definition of the machine lives in a Dockerfile or in `.devcontainer/devcontainer.json`.

They are shorter than you might guess, because they layer on top of other people's definitions:

```
"image": "mcr.microsoft.com/devcontainers/python:1-3.13-bookworm",
```

Both the *Our Story* activity and the *What killed Emi's flowers* activity take place in containers. You can read their `.devcontainer/devcontainer.json` to see exactly how little is in there.

The four kinds of documentation, and the primacy of QUICKSTART.md

The standard classification of the kinds of documentation that good coders use:

- ▶ references
- ▶ tutorials
- ▶ how-tos
- ▶ explanations

One kind of how-to is QUICKSTART.md: instructions for someone else, starting from recreating the environment and ending at a generated output, e.g. a plot.

Keep QUICKSTART.md continuously up to date - know that someone else could run it without you — and you will find yourself at peace.